

Android Port — Implementation Plan

Android Port — Implementation Plan

Project: Onomondo IoT IPA (SGP.32) **Scope:** Add Android as a build target, delivered as both a system daemon and an APK. Configuration is loaded from a JSON file (replacing the Linux command-line flags). The daemon writes the log to a size-limited rotating file; the APK displays the log live while active. **Date:** 2026-08-03

Verdict

The daemon is very feasible. The APK is feasible to build and run, and — because it ships as a **system/signature app on AOSP/OEM firmware** driving the **device's built-in eSIM** — the eUICC-access policy gate that would block an ordinary app is satisfied by the platform signature. Everything else requested (JSON config, rotating log file, on-screen log) is small, self-contained work.

The code side is easy because the codebase is already cleanly layered: all OS-specific surface is confined to three files, and the core is portable C99.

What is already portable

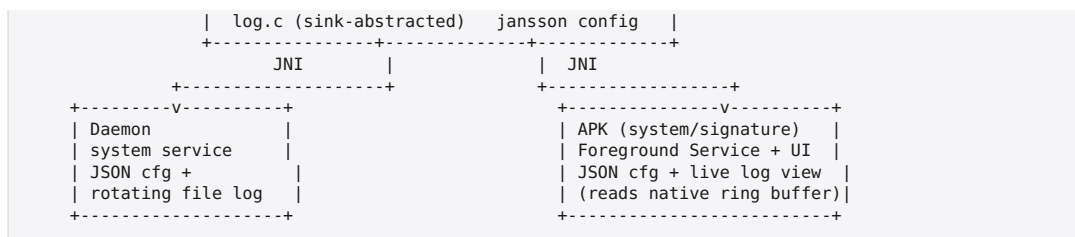
Layer	Files	Android status
Core protocol logic	libipa/ (ES10x, ESipa, procedures)	Pure C99, no file I/O, no threads, no fork/daemon — all confined to main.c. Cross-compiled with the NDK unchanged.
ASN.1 codec	libasn/ (asn1c-generated)	Portable C; _DEFAULT_SOURCE/Bionic quirks are minor.
Crypto	http.c -> OpenSSL	Android's BoringSSL is OpenSSL-family, so it satisfies the CURLOPT_SSL_CTX_FUNCTION requirement from the earlier TLS work (unlike GnuTLS).
HTTP	http.c -> libcurl	Standard NDK build.
JSON	esipa_json.c -> jansson	Already a dependency, so config parsing needs no new library.

The whole core talks to the outside world through two narrow abstractions: ipa_scard_* (5 functions, all void*) and ipa_http*. The core only ever calls ipa_scard_transceive/init/free and ipa_http*; it never touches PC/SC or curl directly. That is the seam we port against.

Target topology

One shared native core (libipa + libasn + http + a new Android scard backend), consumed by two front-ends:

```
+-----+
| libipacore.so  (NDK-built, C99)      |
| libasn . libipa . http(BoringSSL+curl) |
| scard_android.c  <- new backend      |
+-----+
```



Both front-ends reach the eUICC through the same Android telephony path, since the app ships as a system/signature app on AOSP/OEM firmware.

Three design decisions that drive everything

1. eUICC transport = TelephonyManager logical channel (recommended); OMAPI as fallback

As a system app you hold `MODIFY_PHONE_STATE`, so `iccOpenLogicalChannel(AID)` / `iccTransmitApduLogicalChannel(...)` / `iccCloseLogicalChannel()` are available with no SE-side ARA/ARF provisioning to arrange. OMAPI (`android.se.omapi`) is the alternative and maps more cleanly to raw APDUs (`Channel.transmit(byte[])` corresponds directly to `ipa_scard_transceive`), but its access is gated by an Access Rule Applet you would have to provision. Recommendation: TelephonyManager first; keep OMAPI as a compile-time-selectable backend if a device's telephony stack misbehaves.

2. The framework owns the channel — so the core must stop managing it

Both Android APIs perform `MANAGE CHANNEL` and the `ISD-R SELECT` internally, and they police the CLA channel bits. The core currently does both itself (`euicc.c: manage_channel()` sends `INS 0x70`, `select_isd_r()` sends the `SELECT`, and every APDU ORs the channel number into its CLA byte).

Resolution: add a transport capability flag `scard_manages_channel` (queried from the backend). When set, `ipa_euicc_init_es10x()` skips `manage_channel()` / `select_isd_r()`, and the per-APDU CLA channel-bit OR is neutralized (the transport rewrites CLA to the framework-assigned channel). This is a small, contained change at one seam — the raw ES10x transceive logic is untouched, and the Linux PC/SC path stays byte-for-byte unchanged behind the flag.

3. Logging funnels through one function

`ipa_logp` in `log.c` is the only log path, and it currently hardcodes `fprintf(stderr, ...)`. Introduce a pluggable sink: the daemon installs a rotating-file sink, the APK installs a ring-buffer sink the UI polls. No call sites change.

Phase 0 — NDK build

- Use the NDK's CMake toolchain file; parameterize `ABI` / `minSdk` (target `arm64-v8a` + `armeabi-v7a`, `minSdk >= 26` for OMAPI / stable telephony APDU APIs).
- Provide Android builds of dependencies: BoringSSL (satisfies the `CURLLOPT_SSL_CTX_FUNCTION` requirement — OpenSSL-family, unlike GnuTLS), libcurl, jansson. Fetch via an `ExternalProject` superbuild or prebuilts.
- New top-level option `IPA_TARGET_ANDROID` that: selects `scard_android.c` instead of `scard.c`, drops the `pcsc-lite` link, and builds `libipacore.so` (shared) instead of the `ipa` executable.
- Bionic vs glibc: `_DEFAULT_SOURCE` is already project-wide; verify no `<PCSC/*>` / `<wintypes.h>`

leaks in when `IPA_TARGET_ANDROID` is set.

- **Deliverable:** core + net + crypto compile and link into a `.so` for both ABIs. No eUICC yet.

Phase 1 — Android scard backend (the real work)

New file `src/ipa/scard_android.c` implementing the existing 5-function contract in `scard.h`:

Function	Android implementation
<code>ipa_scard_init(reader_num)</code>	JNI up-call to open the logical channel to the ISD-R AID (<code>A0000000559101002...</code> , from <code>select_isd_r</code> in <code>euicc.c</code>). Stores the framework channel + JVM handles in the ctx. <code>reader_num</code> maps to SIM slot index.
<code>ipa_scard_transceive(ctx,res,req)</code>	Marshal <code>req</code> bytes to the JVM; TelephonyManager path decomposes into <code>cla/ins/p1/p2/p3/data</code> and rewrites the CLA channel bits, OMAPI path passes raw; copy response (incl. SW) back into <code>res</code> .
<code>ipa_scard_reset</code> / <code>ipa_scard_atr</code>	Stubs — never called by the core (verified). Reset returns success; ATR returns empty, or <code>Session.getATR()</code> on the OMAPI path.
<code>ipa_scard_free</code>	Close the logical channel, release JNI globals.

Core-side change (single seam): add `scard_manages_channel` capability; make `ipa_euicc_init_es10x()` / `ipa_euicc_close_es10x()` and the CLA channel-bit OR conditional on it. Guard with the flag so the Linux PC/SC path is unchanged.

JNI bridge: `scard_jni.c` (native) + a small Kotlin/Java `EuiccChannel` class wrapping `TelephonyManager`. Handle: exception propagation (Java exception -> `-EIO`), thread attach/detach (the poll loop runs on a native thread, so `AttachCurrentThread`), and slot/subscription selection.

Risk to validate early: `iccTransmitAduLogicalChannel`'s CLA channel-bit handling and its treatment of GET RESPONSE / 61xx chaining — confirm the core's `recv_es10x_block` GET RESPONSE loop survives when the framework auto-handles 61xx. This is the single highest-risk item; prototype it against real hardware before building the rest.

Phase 2 — JSON configuration

- New entry module `ipa_config_json.c`: parse a config file with `jansson` (already a dependency) into `struct ipa_config` plus the auxiliary paths currently held as locals in `main.c` (`nvstate` path, `initial-eim-config` path, `one-pkg-only`, `memory-reset`).
- Key-per-CLI-flag mapping (all fields already flat in `ipad.h`): `tac`, `preferred_eim_id`, `reader_num`, `euicc_channel`, `eim_cabundle`, `eim_disable_ssl`, `eim_disable_ssl_verif`, `esipa_req_retries`, `esipa_binding`, `iot_euicc_emu_enabled`, plus `nvstate_path`, `initial_eim_cfg_path`, and a `log_block` (Phase 3).
- Validation + defaults mirror `main.c`'s current defaults; on missing/invalid file, fail with a clear log line.
- `main.c` stays the Linux/PC-SC entry; the daemon and APK call a new shared `ipa_run_from_config(const char *json_path)`. Keeps CLI Linux behavior intact.

CLI flag -> JSON key mapping

CLI flag	JSON key	Notes
<code>-t TAC</code>	<code>tac</code>	hex string
<code>-e eimId</code>	<code>preferred_eim_id</code>	optional

-r N	reader_num	SIM slot on Android
-c N	euicc_channel	ignored when transport manages the channel
-f PATH	initial_eim_cfg_path	
-m	euicc_memory_reset	boolean
-n PATH	nvstate_path	
-y NUM	esipa_req_retries	
-C	eim_cabundle	
-S	eim_disable_ssl	boolean
-I	eim_disable_ssl_verif	boolean
-E	iot_euicc_emu_enabled	boolean
-l	one_euicc_pkg_only	boolean
(new)	log.max_size_bytes	rotating-file sink
(new)	log.max_files	rotating-file sink

Phase 3 — Logging: sink abstraction + rotation

- In `log.c`: replace the hardcoded `fprintf(stderr, ...)` in `ipa_logp` with a registered sink `void (*)(const char *line, size_t len)` (default sink = `stderr`, so Linux is unchanged). Add `ipa_log_set_sink(...)`.
- Rotating-file sink** (daemon): size-tracked writer honoring `log.max_size_bytes` + `log.max_files` from JSON; rename `foo.log` -> `foo.log.1` ... on threshold. Contained in a new `log_file_sink.c`.
- Ring-buffer sink** (APK): fixed-capacity in-memory buffer with a JNI getter the UI polls (or a callback pushed to the JVM). New `log_ring_sink.c`.
- Also route the `printf` status banner in the entry path through the log so the APK sees it too (small cleanup; `print_help` stays CLI-only).

Phase 4 — Daemon packaging

- Build `libipacore.so` + a tiny native `main (ipad_android.c)` -> `ipad` executable, launched as an `init .rc` service (system image) or an app-owned foreground service's child. Runs `ipa_run_from_config("/data/.../config.json")` and installs the rotating-file sink.
- Signal handling: keep the `SIGUSR1` -> `running=false` graceful-stop from `main.c`.
- SELinux: a system daemon needs a policy domain allowing telephony access — flag for the AOSP/OEM integrator (sepolicy `.te` additions).

Phase 5 — APK

- Native:** the same `libipacore.so` via JNI. A `NativeBridge` (Kotlin) exposes `start(configPath)`, `stop()`, `pollLog()`.
- Config:** ship/generate `config.json` in app storage; a settings screen edits it against the same JSON schema.
- eUICC access:** `TelephonyManager` calls happen in Kotlin and are handed to native via the Phase-1 JNI channel object — as a signature/system app the `MODIFY_PHONE_STATE` grant is

satisfied by the platform signature; declare it in the manifest and get the app onto the system image / signed with the platform key.

- **UI:** a foreground `Service` runs the poll loop off the main thread; a Compose screen tails the ring-buffer sink and renders log lines live while active. Start/stop controls map to the service.
- **Manifest/permissions:** `MODIFY_PHONE_STATE` , `FOREGROUND_SERVICE` , `internet`; privileged-permission allow-list entry for the system-app install.

Risk register

Risk	Severity	Mitigation
Framework channel management vs. core's self-managed channel/SELECT + 61xx GET RESPONSE handling	High	Prototype Phase-1 transceive against real hardware first; gate core changes behind <code>scard_manages_channel</code> so Linux is untouched.
<code>iccTransmitApuLogicalChannel</code> masking/rewriting CLA bits	Medium	Neutralize the core's CLA channel OR in the Android transport; test SW=9000 round-trips.
SELinux denials for a system daemon reaching telephony	Medium	Provide sepolicy additions; OEM integrator applies.
BoringSSL/curl/jansson NDK build friction	Low	Well-trodden; superbuild.
JNI threading (poll loop on native thread)	Low	<code>AttachCurrentThread</code> in the transport; cache <code>JavaVM*</code> .

Effort recap

- **Phase 0:** small-moderate.
- **Phase 1:** moderate-large + the one hardware-validation spike (dominant on the critical path).
- **Phases 2-3:** ~1 day each.
- **Phase 4:** moderate (mostly integration / sepolicy).
- **Phase 5:** moderate (the Android UI/service is the bulk).

The Linux/PC-SC build stays fully intact throughout — every Android change is behind `IPA_TARGET_ANDROID` or a runtime capability flag.

Critical-path recommendation: do the Phase-1 transceive spike against a real eSIM before committing to the rest. The channel-management / GET-RESPONSE behavior of the Android telephony APIs is the only thing here that can force a redesign, and it is cheap to de-risk up front.